Autores: [Diego Alexander Niño Laguado, Carlos Andrés Gutiérrez Rodríguez, Germán Yesid Vanegas Cely, Juan Esteban Fiagá Suárez]
Profesor: Urbano Eliécer Gómez Prada

Contenido

1 Contexto 1
2 Análisis 1
2.1 Estándares / Criterios de Calidad 2
2.2 Requerimientos 2
2.2.1 Requerimientos Funcionales (RF) 2
2.2.2 Requerimientos No Funcionales (RNF) 2
2.2.3 Burndown Chart 3
2.2.4 Trello 3
2.3 Puntos de función 3
2.3.1 Entradas Externas (EI) 3
2.3.2 Salidas Externas (EO) 3
2.3.3 Consultas Externas (EQ) 3
2.3.4 Archivos Lógicos Internos (ILF) 3
2.3.5 Archivos de Interfaz Externa (EIF) 3
2.3.6 Resumen de Puntos de Función 3
2.4 Plan de pruebas 3
2.4.1 Estrategia y Tipos de Pruebas: 3
2.4.2 Casos de Prueba Principales 4
3 Diseño 4
3.1 Casos de uso general 5
3.2 Actividades 5
3.3 Vistas de Usuario 6
3.4 BD 7
3.5 Arquitectura (Tecnologías) 7
4 Implementación 7
4.1 Entorno de Desarrollo y Configuración 8
4.2 Módulos Implementados 8
5 Conclusiones 8
6 Trabajo futuro 8
7 Fuentes 9

Tabla de Figuras

Figura 1. Caso de uso general 4
Figura 2. Diagrama de secuencia para registro de donación 5
Figura 3. Vista de usuario página inicio con informes generales 5
Figura 4. Vista de usuario registro de donación 5
Figura 5. Vista de usuario resultados de informes 6
Figura 6. Diagrama base de datos banco de alimentos 6

1 Contexto

Los bancos de alimentos y comedores comunitarios cumplen un rol social prioritario al canalizar insumos hacia poblaciones vulnerables. No obstante, la gestión operativa mediante herramientas no centralizadas (como hojas de cálculo) genera pérdida de productos por caducidad y falta de trazabilidad en las entregas.

SIGA (Sistema Inteligente de Gestión de Alimentos) aborda esta problemática integrando una arquitectura de base de datos relacional para el registro de usuarios, donantes, beneficiarios, categorías, productos, donaciones y solicitudes. La solución busca automatizar el seguimiento de inventarios y habilitar un mecanismo de control preventivo mediante la detección de alertas de vencimiento.

2 Análisis

El análisis del sistema abarca el ciclo de vida operacional desde el ingreso de donaciones hasta la atención de solicitudes de apoyo social:

- **Gestión de Actores y Autenticación:** Se gestionan perfiles centralizados en **Usuarios**, vinculando roles con el control de donaciones y atención de solicitudes.
- **Flujo de Ingreso de Donaciones:** Regula el registro de proveedores externos en **Donantes** y sus entregas en **Donaciones**.
- **Clasificación y Gestión de Inventario:** Indexación de insumos en **Productos** parametrizados por su **Categorias** y unidad de medida.
- **Procesamiento de Solicitudes:** Las necesidades de las familias registradas en **Beneficiarios** se formalizan a través de

Solicitudes vinculadas a productos específicos y cantidades requeridas.

- **Monitoreo Preventivo:** Registro de estados críticos en **Alertavencimiento** para prevenir pérdidas de inventario.

2.1 Estándares / Criterios de Calidad

ISO/IEC 25010 (Calidad de Software):

- **Adecuación Funcional:** Integridad en la trazabilidad relacional entre donantes, donaciones, beneficiarios, productos y solicitudes.
- **Usabilidad:** Interfaz web intuitiva orientada al personal de bodega y gestión de entregas.
- **Integridad y Consistencia de Datos:** Reglas de integridad referencial ACID en la base de datos para prevenir inconsistencias.
- **Estándar ISO/IEC/IEEE 29148 (Especificación de Requisitos):**
- Definición rigurosa de requisitos funcionales y no funcionales basados en las entidades del diagrama.
- **Criterios Operativos de Calidad:**
- **Efectividad en Trazabilidad:** Capacidad de asociar cada producto solicitado al beneficiario y su núcleo familiar.
- **Tiempo de Respuesta:** Consultas optimizadas con claves primarias (PK) y foráneas (FK).

2.2 Requerimientos

2.2.1 Requerimientos Funcionales (RF)

- **RF01 - Gestión de Usuarios:** El sistema debe permitir el registro, autenticación y asignación de roles (rol) en la entidad Usuarios para controlar el acceso a los módulos operativos.
- **RF02 - Registro de Donantes y Donaciones:** El sistema debe capturar la información del donante en Donantes y registrar la donación en Donaciones asociando la fecha y el estado correspondiente.
- **RF03 - Catalogación de Productos y Categorías:** El sistema debe gestionar las categorías de alimentos en Categorias y parametrizar cada producto en Productos especificando su unidadMedida.
- **RF04 - Registro de Beneficiarios:** El sistema debe almacenar la información de los beneficiarios en Beneficiarios, incluyendo el atributo numPersonasHogar para priorizar el apoyo según el núcleo familiar.
- **RF05 - Gestión de Solicitudes de Alimentos:** El sistema debe permitir registrar solicitudes en la entidad Solicitudes, vinculando el beneficiario (beneficiarioId), el producto solicitado (prodId), la cantidadSolicitada y el estado de la transacción.
- **RF06 - Monitoreo de Alertas de Vencimiento:** El sistema debe registrar y desplegar notificaciones en Alertavencimiento, evaluando el nivelUrgencia para prevenir el desperdicio de insumos.

2.2.2 Requerimientos No Funcionales (RNF)

- **RNF01 - Integridad Referencial:** La base de datos PostgreSQL debe aplicar restricciones de clave primaria (PK) y clave foránea (FK) entre todas las tablas para garantizar la consistencia relacional.
- **RNF02 - Usabilidad y Diseño Responsive:** La interfaz web debe ser intuitiva y adaptable a dispositivos móviles/tablets para facilitar el trabajo del personal en bodega.
- **RNF03 - Desempeño:** Las consultas relacionales de disponibilidad y filtrado de productos deben ejecutarse en un tiempo inferior a 2 segundos bajo carga estándar.

2.2.3 Burndown Chart

2.2.4 Trello

2.3 Puntos de función

Para realizar la estimación del tamaño del software se utilizó la metodología de **Puntos de Función No Ajustados (PFNA)** analizando las entidades de la base de datos:

2.3.1 Entradas Externas (EI)

- Registro / Edición de Usuarios (Media: 4 PF)
- Registro / Edición de Donantes (Baja: 3 PF)
- Registro / Edición de Beneficiarios (Baja: 3 PF)
- Registro / Edición de Productos (Baja: 3 PF)
- Creación de Solicitudes (Media: 4 PF)

Total EI = 17 PF

2.3.2 Salidas Externas (EO)

- Reporte de Solicitudes procesadas (Media: 5 PF)
- Generación de alertas en Alertavencimiento (Alta: 7 PF)

Total EO = 12 PF

2.3.3 Consultas Externas (EQ)

- Consulta de stock de Productos por Categorias (Baja: 3 PF)
- Historial de Donaciones por Donantes (Baja: 3 PF)
- Consulta de Beneficiarios (Baja: 3 PF)

Total EQ = 9 PF

2.3.4 Archivos Lógicos Internos (ILF)

- Grupos de datos principales: Usuarios, Donantes, Beneficiarios, Productos, Donaciones, Solicitudes, Alertavencimiento (7 ILF $\times$ 7 PF promedio = 49 PF)

Total ILF = 49 PF

2.3.5 Archivos de Interfaz Externa (EIF)

- Tabla de referencia Categorias (Baja: 5 PF)

Total EIF = 5 PF

2.3.6 Resumen de Puntos de Función

- Total Puntos de Función No Ajustados (PFNA): 17 + 12 + 9 + 49 + 5 = 92 PF.
- Factor de Ajuste de Complejidad (VAF): Se define un valor de VAF de 0.95, considerando la complejidad técnica de un entorno web relacional con validaciones de negocio.
- Puntos de Función Ajustados (PFA): PFA = PFNA * VAF = 92 * 0.95 = 87.4 PF.

2.4 Plan de pruebas

2.4.1 Estrategia y Tipos de Pruebas:

- **Pruebas Unitarias:** Validación de componentes individuales y métodos clave en el backend (ej. métodos de validación de correo de usuario, cálculo de fechas de expiración en las alertas).
- **Pruebas de Integración:** Verificación de las interacciones entre los componentes y la base de datos PostgreSQL (ej. verificar que al registrar una solicitud se cree el registro correcto con sus llaves foráneas beneficiarioId y prodId).
- **Pruebas Funcionales (Caja Negra):** Validación del flujo completo de uso desde la interfaz de usuario (ej. flujo de registro de un nuevo donante o creación de una solicitud).
- **Pruebas de Integridad de Base de Datos:** Verificación de que las restricciones de claves primarias (PK), claves foráneas (FK) y reglas de no nulidad se cumplan rigurosamente.

2.4.2 Casos de Prueba Principales

ID Prueba	Módulo / Entidad	Descripción del Escenario	Resultado Esperado
CP-01	Usuarios	Registrar usuario con correo en formato inválido o repetido.	El sistema rechaza el registro y muestra un mensaje de validación.
CP-02	Donaciones	Registrar una donación vinculada a un donanteId existente.	Se crea el registro exitosamente en la tabla Donaciones con la fecha actual.
CP-03	Productos / Categorias	Crear un producto asociándole un categoriaId no existente.	La base de datos rechaza la transacción violando la restricción de FK.
CP-04	Solicitudes	Registrar una solicitud para un beneficiario indicando una cantidad válida de producto.	La solicitud queda registrada en estado "Pendiente" con sus referencias correctas.
CP-05	Alerta-vencimiento	Consultar el panel de alertas de vencimiento.	Se listan los registros de alertas ordenados por nivelUrgencia.

3 Diseño

3.1 Casos de uso general

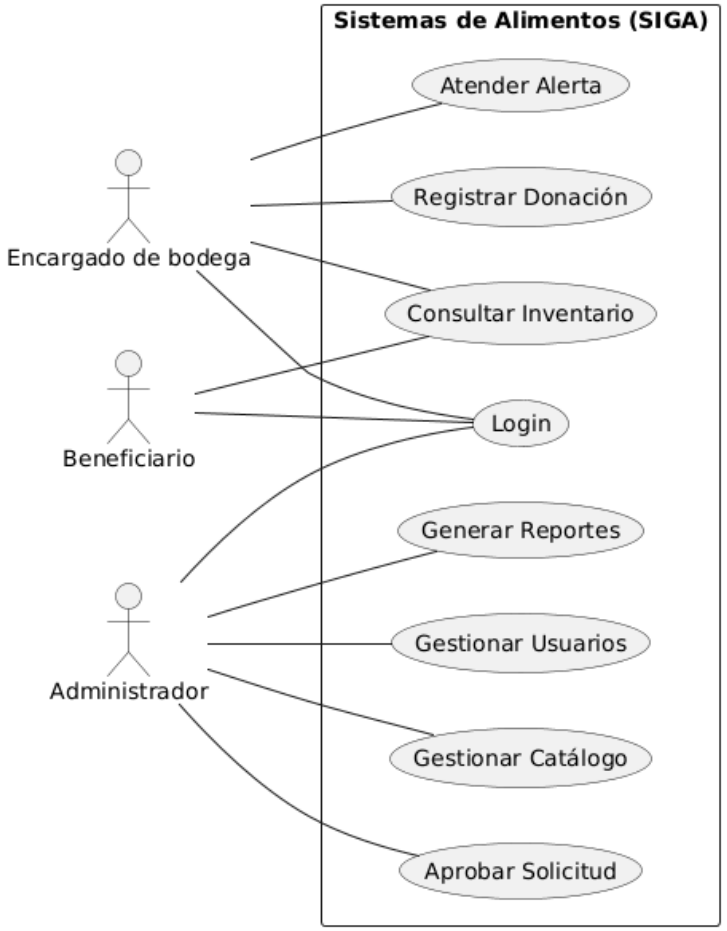


Figura 1. Caso de uso general

3.2 Actividades

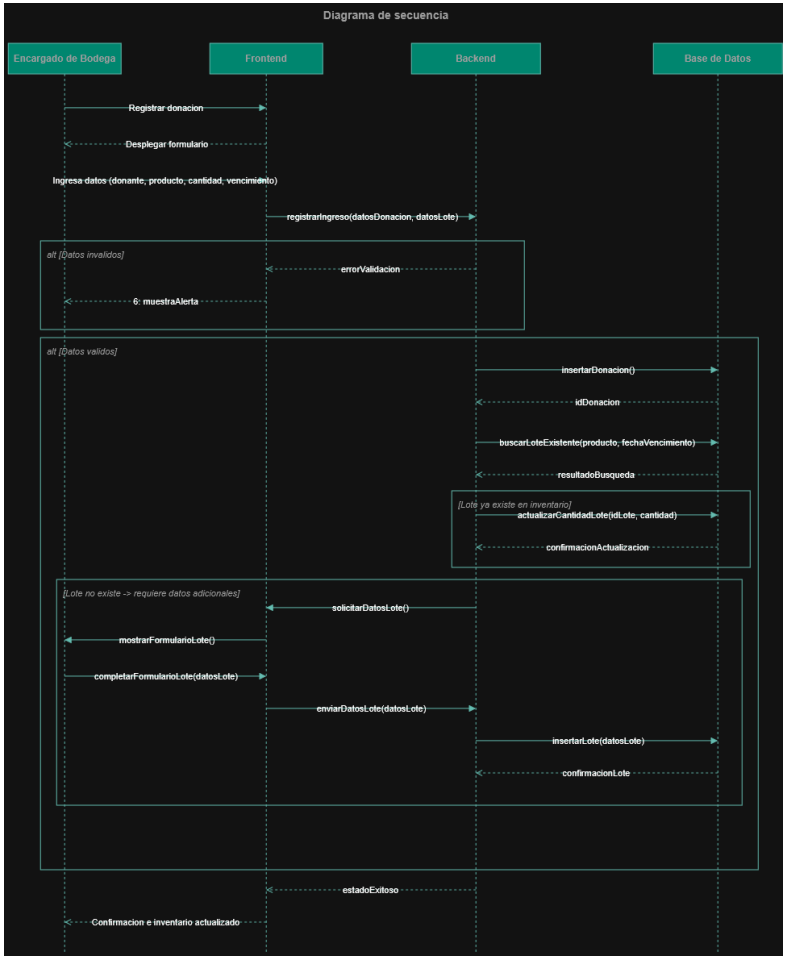


Figura 2. Diagrama de secuencia para registro de donación

3.3 Vistas de Usuario

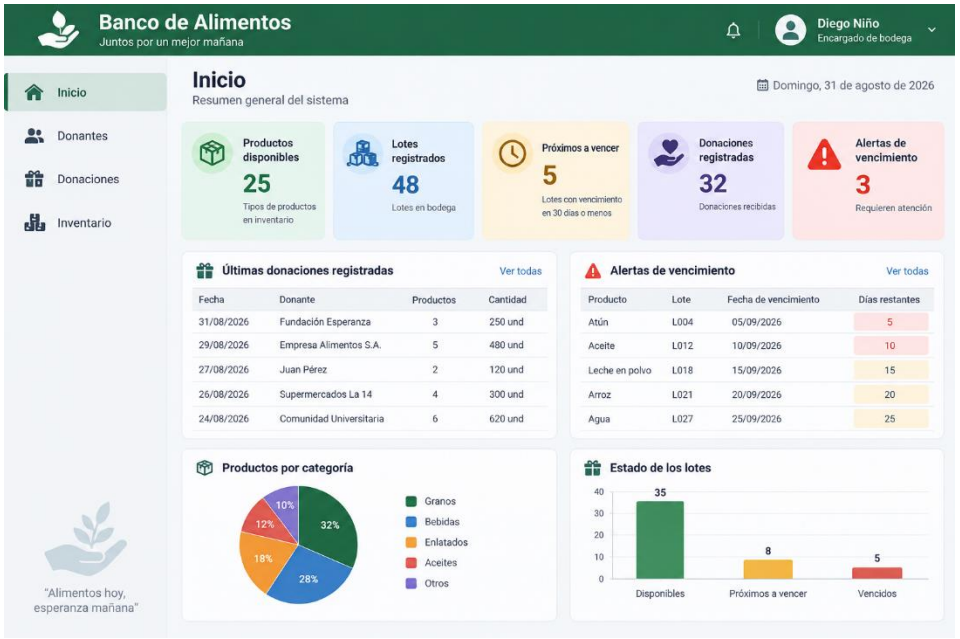


Figura 3. Vista de usuario página inicio con informes generales

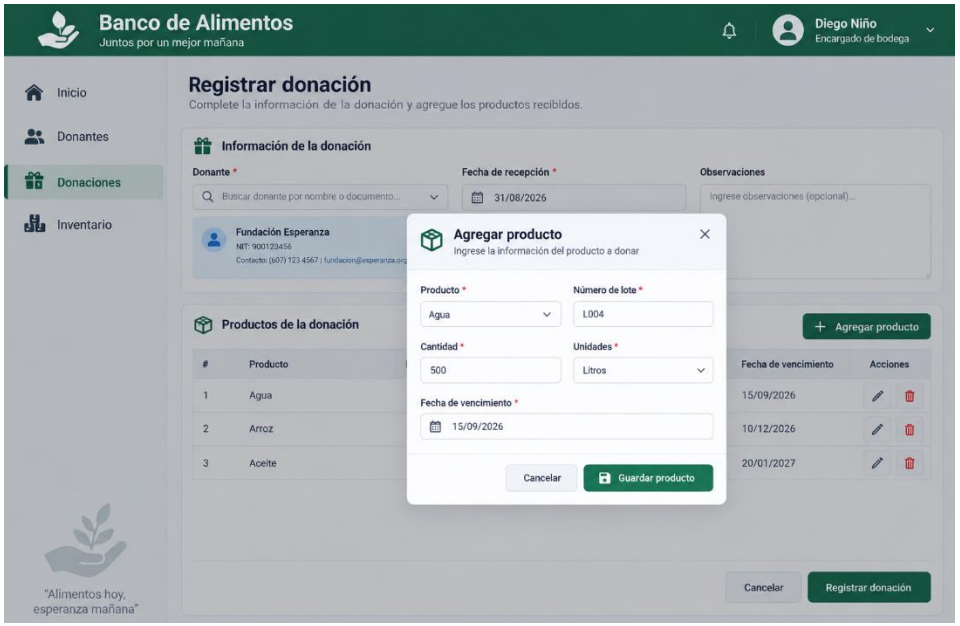


Figura 4. Vista de usuario registro de donación

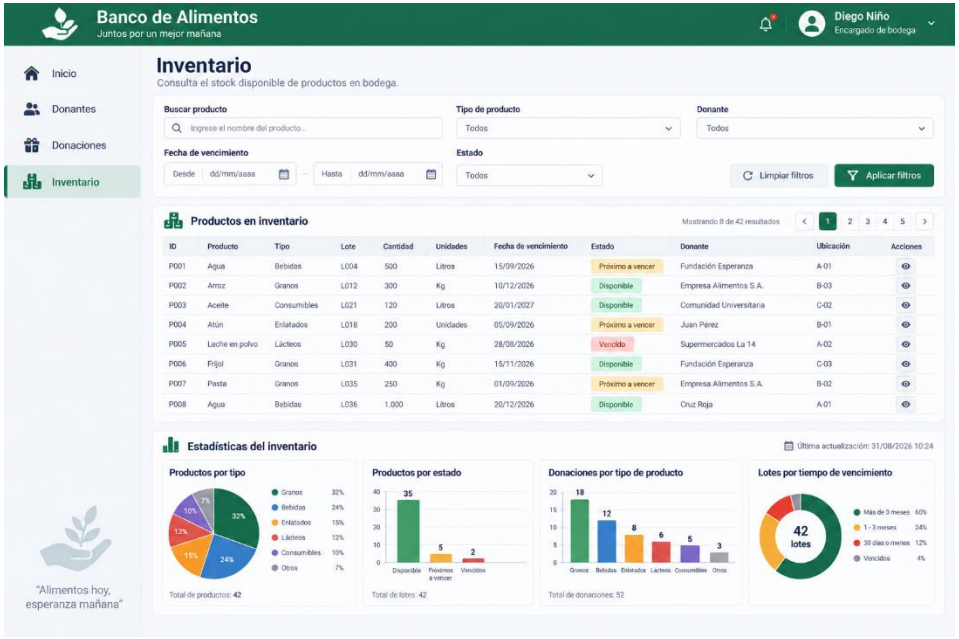


Figura 5. Vista de usuario resultados de informes

3.4 BD

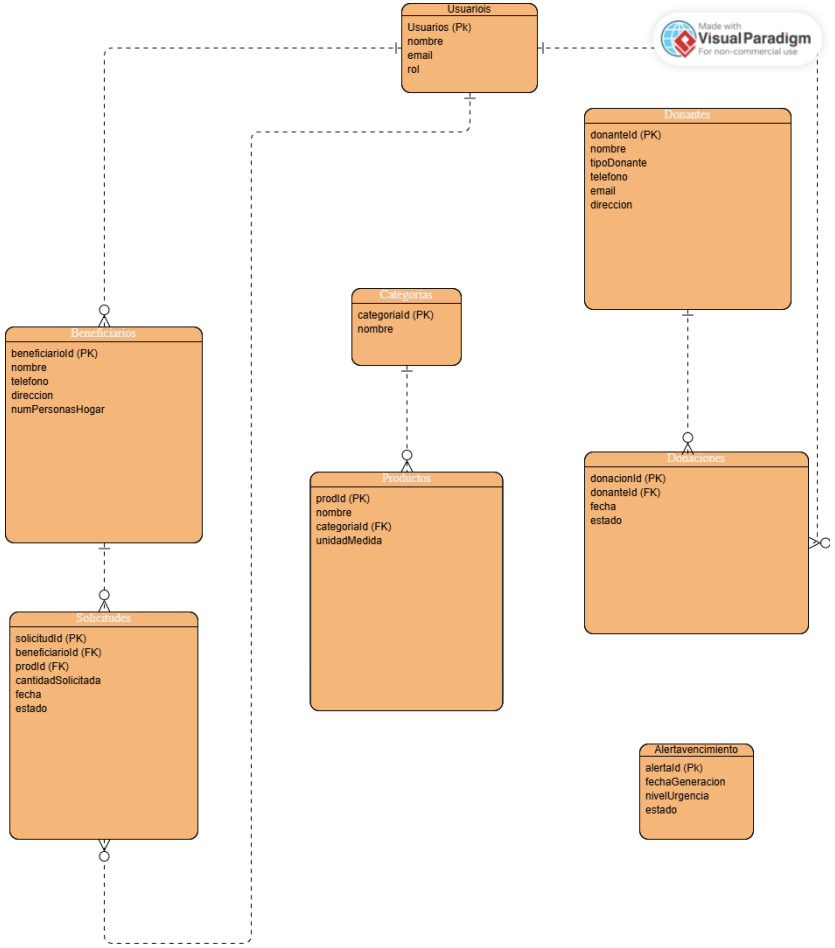


Figura 6. Diagrama base de datos banco de alimentos

3.5 Arquitectura (Tecnologías)

El sistema **SIGA** adopta una arquitectura cliente-servidor desacoplada basada en capas, diseñada para garantizar la escalabilidad, mantenibilidad y separación clara de responsabilidades:

- **Capa de Presentación (Frontend):**

- **Tecnología:** React / HTML5, CSS3 y JavaScript (ES6+).
- **Descripción:** Interfaz web responsive e interactiva que facilita la captura de datos en bodegas desde computadores o dispositivos móviles. Consume los servicios expuestos por el backend mediante peticiones HTTP asíncronas (REST API).

- **Capa de Logica de Negocio (Backend):**
  - **Tecnología:** Java con Spring Boot / Node.js (NestJS).
  - **Descripción:** Encargada de procesar las reglas de negocio, validar la autenticación y roles de los usuarios, calcular los niveles de urgencia en la entidad Alertavencimiento y gestionar el flujo de las Solicitudes enviadas por los beneficiarios.
- **Capa de Persistencia de Datos (Base de Datos):**
  - **Tecnología:** PostgreSQL.
  - **Descripción:** Motor de base de datos relacional encargado de almacenar y estructurar las entidades Usuarios, Donantes, Beneficiarios, Categorias, Productos, Donaciones, Solicitudes y Alertavencimiento. Garantiza la integridad referencial y las propiedades ACID en cada transacción.
- **Control de Versiones y Despliegue:**
  - **Herramientas:** Git y GitHub para la gestión del código fuente, integración continua y trabajo colaborativo del equipo de desarrollo.

4 Implementación

La fase de implementación comprende la traducción de los modelos de diseño en artefactos de software funcionales y la puesta en marcha del entorno de ejecución.

4.1 Entorno de Desarrollo y Configuración

- **IDE / Editores:** Visual Studio Code / IntelliJ IDEA para la codificación y gestión de scripts.
- **Gestor de Base de Datos:** PostgreSQL con la ejecución del script DDL que define las 8 tablas primarias, sus respectivas claves primarias (PK), claves foráneas (FK) y restricciones de integridad.
- **Construcción de la API REST:** Definición de controladores y endpoints estructurados bajo los estándares HTTP (GET, POST, PUT, DELETE) para gestionar el ciclo de vida de los datos de insumos, solicitudes y usuarios.

4.2 Módulos Implementados

- **Módulo de Autenticación y Usuarios:** Control de acceso mediante roles (Administrador, Encargado de Bodega, Beneficiario) utilizando la entidad Usuarios.
- **Módulo de Gestión de Inventario y Donaciones:** Registro continuo de proveedores en Donantes, catalogación en Productos/Categorias y registro de ingresos en Donaciones.
- **Módulo de Solicitudes y Alertas:** Captura de requerimientos de consumo en Solicitudes y habilitación del panel de monitoreo de insumos en Alertavencimiento.

5 Conclusiones

- El desarrollo e implementación del **Sistema Inteligente de Gestión de Alimentos (SIGA)** permite centralizar y digitalizar la operación en bancos de alimentos, reemplazando esquemas de registro manuales u ofimáticos por una plataforma estructurada y confiable.

- El diseño simplificado del modelo de datos relacional en PostgreSQL (compuesto por las entidades Usuarios, Donantes, Beneficiarios, Categorias, Productos, Donaciones, Solicitudes y Alertavencimiento) garantiza la integridad referencial y optimiza el tiempo de respuesta en la trazabilidad de insumos.
- La incorporación del módulo Alertavencimiento provee un mecanismo de control preventivo sobre el estado de los insumos, ayudando a mitigar el desperdicio en bodega y agilizando la toma de decisiones para la distribución prioritaria de productos.
- La correcta definición de requerimientos funcionales y no funcionales, junto con la especificación de un plan de pruebas estructurado, asegura que la solución tecnológica cumpla con estándares de calidad en adecuación funcional, usabilidad y fiabilidad para el entorno operativo de bodega.

6 Trabajo futuro

- **Implementación de la trazabilidad granular por lotes y regla FEFO (*First Expired, First Out*):** Reincorporar la gestión e inventario por lotes individuales (id_lote) con fechas explícitas de vencimiento por cada producto recibido, permitiendo automatizar de forma estricta la salida de insumos según su cercanía de caducidad.
- **Algoritmos de asignación equitativa y priorización por vulnerabilidad:** Desarrollar un motor de recomendación o reglas de scoring que evalúe la urgencia y el número de personas en el hogar (numPersonasHogar) para automatizar la distribución justa entre distintas entidades beneficiarias.
- **Optimización logística y georreferenciación:** Integrar módulos de ruteo y mapas para la recolección de donaciones grandes y la entrega eficiente en territorio a comedores comunitarios y beneficiarios.
- **Desarrollo de aplicación móvil nativa o PWA:** Diseñar una versión web progresiva o app móvil orientada al personal de bodega y donantes para el escaneo rápido de insumos mediante códigos de barras o QR.
- **Integración API con sistemas externos:** Habilitar interoperabilidad mediante servicios web con sistemas ERP o de inventario de grandes cadenas de supermercados e industrias donantes para la sincronización automática de insumos.

7 Fuentes

1. Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Addison-Wesley / Pearson Educación.
    a. *Aporte al proyecto:* Guía para la especificación de requerimientos funcionales y no funcionales (IEEE 830 / ISO/IEC/IEEE 29148), modelos de ciclo de vida de desarrollo ágil y diseño de arquitectura de software en capas.
2. Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico* (9.ª ed.). McGraw-Hill Education.
    a. *Aporte al proyecto:* Metodología de estimación de tamaño de software mediante Puntos de Función (PF), cálculo de Puntos de Función No Ajustados (PFNA) y gestión del plan de pruebas (pruebas unitarias, de integración y funcionales de caja negra).
3. Organización Internacional de Normalización (ISO) / Comisión Electrotécnica Internacional (IEC). (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011).
    a. *Aporte al proyecto:* Marco de referencia para los criterios de calidad del sistema SIGA, evaluando atributos de adecuación funcional, usabilidad, fiabilidad e integridad de datos.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill Education.

a. *Aporte al proyecto:* Diseño e implementación del modelo relacional en PostgreSQL, definición de claves primarias (PK), claves foráneas (FK), restricciones de integridad referencial y propiedades ACID en transacciones de inventario y solicitudes.

5. Schwaber, K., & Sutherland, J. (2020). *La Guía de Scrum: La Guía Definitiva para Scrum: Las Reglas del Juego*. Scrum.org.
    a. *Aporte al proyecto:* Planificación del cronograma por Sprints, estructuración del Backlog del producto, uso del tablero visual en Trello y seguimiento de la velocidad del equipo mediante la gráfica de trabajo pendiente (Burndown Chart).